Prep 1: Imports and Data Load.

```python
# Imports
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.feature_selection import SelectKBest, f_classif, chi2
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report, roc_curve, auc

# Load Cleaned Dataset
df = pd.read_csv('breast-cancer-updated.csv')
```

Prep 2: Split features and Label

```python
# Split features and label
X = df.drop(columns=['diagnosis'])
y = df['diagnosis']
```

Step 1: Feature Selection Part 1

```python
# Step 1: Feature Selection (Fisher Score and Chi-Square)

# Standardize for Fisher Score
scaler_std = StandardScaler()
X_scaled_std = scaler_std.fit_transform(X)

# Normalize for Chi-Square
scaler_minmax = MinMaxScaler()
X_scaled_minmax = scaler_minmax.fit_transform(X)

# Fisher Score
def fisher_score(X, y):
    classes = np.unique(y)
    scores = []
    for i in range(X.shape[1]):
        numerator = 0
        denominator = 0
        for c in classes:
            idx = (y == c)
            class_mean = np.mean(X[idx, i])
            overall_mean = np.mean(X[:, i])
            numerator += sum(idx) * (class_mean - overall_mean) ** 2
            denominator += np.sum((X[idx, i] - class_mean) ** 2)
        scores.append(numerator / (denominator + 1e-6))
    return np.array(scores)

fisher_scores = fisher_score(X_scaled_std, y)
fisher_df = pd.DataFrame({
    'Feature': X.columns,
    'FisherScore': fisher_scores
}).sort_values(by='FisherScore', ascending=False)

# Print Fisher Score Results
print("Top Features by Fisher Score:")
print(fisher_df.head(10))

# Chi-Square
chi2_scores, _ = chi2(X_scaled_minmax, y)
chi2_df = pd.DataFrame({
    'Feature': X.columns,
    'ChiSquare': chi2_scores
}).sort_values(by='ChiSquare', ascending=False)

# Print Chi-Square Results
print("\nTop Features by Chi-Square:")
print(chi2_df.head(10))
```

```
Top Features by Fisher Score:
                 Feature  FisherScore
27  concave points_worst     1.700856
22       perimeter_worst     1.583676
7    concave points_mean     1.519711
20          radius_worst     1.518134
2         perimeter_mean     1.229692
23            area_worst     1.166843
0            radius_mean     1.141060
3              area_mean     1.010689
6         concavity_mean     0.941434
26       concavity_worst     0.770180

Top Features by Chi-Square:
                 Feature  ChiSquare
7    concave points_mean  52.405743
27  concave points_worst  46.341648
6         concavity_mean  46.186395
23            area_worst  35.043882
22       perimeter_worst  34.438091
20          radius_worst  34.124937
26       concavity_worst  31.563031
3              area_mean  29.328594
2         perimeter_mean  26.528902
0            radius_mean  24.897293
```

Findings:

First, I applied **Fisher Score and Chi-Square** algorithms to rank feature importance in the breast cancer dataset. Fisher Score identified **concave points_worst, perimeter_worst, concave points_mean, and radius_worst** as the top features, while Chi-Square ranked **concave points_mean, concave points_worst, and concavity_mean** highest. Both methods showed **strong overlap, with features like concave points_worst, concave points_mean, perimeter_worst, and area_worst consistently appearing at the top**. Based on these results, I selected the Top 10 features to create a reduced dataset for retraining the model in Part 4.

<u>Step 2: Feature Selection Part 2</u>

```python
# Merge Results
merged_df = fisher_df.merge(chi2_df, on='Feature')

# Select Top 10 Features
selected_features = merged_df.sort_values(by=['FisherScore', 'ChiSquare'], ascending=False).head(10)['Feature'].tolist()

# Print Selected Top 10 Features
print("\nFinal Selected Top 10 Features (after merging Fisher and Chi-Square):")
for i, feat in enumerate(selected_features, 1):
    print(f"{i}. {feat}")
```

```
Final Selected Top 10 Features (after merging Fisher and Chi-Square):
1. concave points_worst
2. perimeter_worst
3. concave points_mean
4. radius_worst
5. perimeter_mean
6. area_worst
7. radius_mean
8. area_mean
9. concavity_mean
10. concavity_worst
```

Findings:

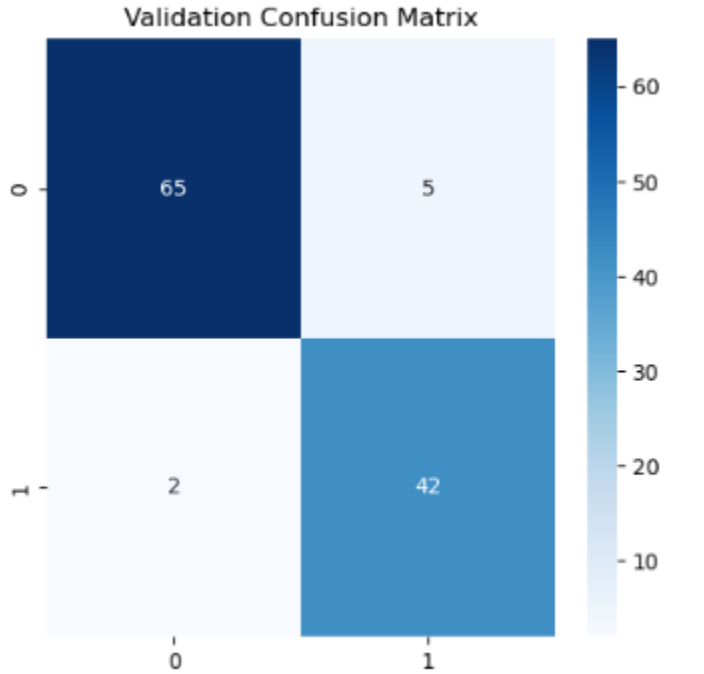
After running Fisher Score and Chi-Square separately, I merged the results to combine the strengths of both selection methods. By sorting based on both Fisher Score and Chi-Square rankings, I was able to generate a consolidated list and select the Top 10 most important features. **These final features were concave points_worst, perimeter_worst, concave points_mean, radius_worst, perimeter_mean, area_worst, radius_mean, area_mean, concavity_mean, and concavity_worst.** These features consistently ranked highly across both scoring methods, reflecting their strong predictive power for distinguishing between benign and malignant tumors. Selecting this reduced feature set ensured that the model focused on the most informative attributes while minimizing dimensionality, which could **help reduce overfitting and improve generalization in the retrained model.**

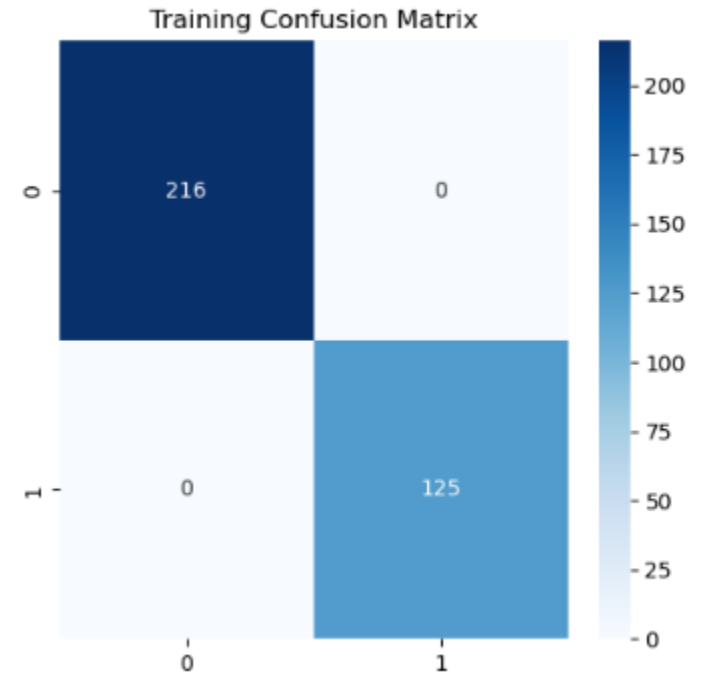
Step 3: Create New Dataset with Selected Features and Train Model

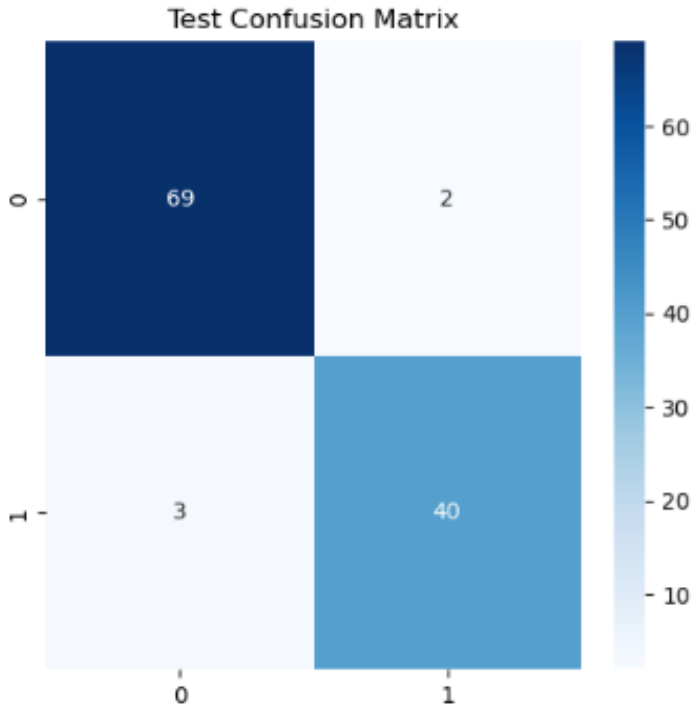
[3]:
```
# Step 2: Create New Dataset with Selected Features and Train Model

# Create new dataset with Top 10 Features
X_selected = df[selected_features]

# Split the data into training, validation, and testing sets
X_train_val, X_test, y_train_val, y_test = train_test_split(X_selected, y, test_size=0.2, random_state=42)
X_train, X_val, y_train, y_val = train_test_split(X_train_val, y_train_val, test_size=0.25, random_state=42)

# Train RandomForestClassifier
rf = RandomForestClassifier(random_state=42)
rf.fit(X_train, y_train)
```

After selecting the Top 10 features, I created a new reduced dataset and split it into training (60%), validation (20%), and testing (20%) subsets. A RandomForestClassifier was then retrained on the new dataset using the same approach as in Part 3.

4: Evaluate Model with Confusion Matrices







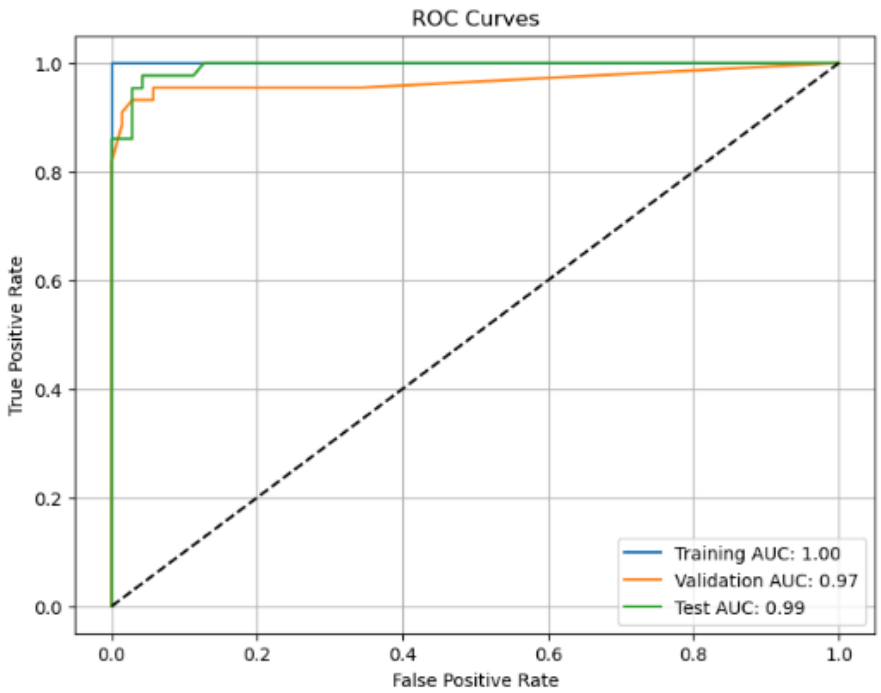
Findings:

After retraining the model on the reduced feature set, I evaluated its performance by generating confusion matrices for the training, validation, and testing sets. The training set confusion matrix showed perfect classification, **with 216 true negatives and 125 true positives**, and no misclassifications. On the validation set, the model made a small number of errors, with **65 true negatives, 42 true positives, 5 false positives, and 2 false negatives**. The test set confusion matrix showed similarly strong results, **with 69 true negatives, 40 true positives, 2 false positives, and 3 false negatives**. These findings indicate that the model generalized well to unseen data, though a small number of malignant tumors were still misclassified as benign on the validation and test sets, which could be critical in a medical context. Overall, the confusion matrices demonstrate that reducing the feature set did not significantly harm classification performance.

Step 5: Calculate Performance Metrics and Plot the ROC curves

```
# Step 4: Calculate Performance Metrics and Plot ROC Curves

# Classification Reports
print("Training Classification Report:")
print(classification_report(y_train, y_train_pred))
print("Validation Classification Report:")
print(classification_report(y_val, y_val_pred))
print("Test Classification Report:")
print(classification_report(y_test, y_test_pred))

# ROC Curves and AUC Scores
probs_train = rf.predict_proba(X_train)[:, 1]
probs_val = rf.predict_proba(X_val)[:, 1]
probs_test = rf.predict_proba(X_test)[:, 1]

fpr_train, tpr_train, _ = roc_curve(y_train, probs_train)
roc_auc_train = auc(fpr_train, tpr_train)

fpr_val, tpr_val, _ = roc_curve(y_val, probs_val)
roc_auc_val = auc(fpr_val, tpr_val)

fpr_test, tpr_test, _ = roc_curve(y_test, probs_test)
roc_auc_test = auc(fpr_test, tpr_test)

plt.figure(figsize=(8,6))
plt.plot(fpr_train, tpr_train, label=f'Training AUC: {roc_auc_train:.2f}')
plt.plot(fpr_val, tpr_val, label=f'Validation AUC: {roc_auc_val:.2f}')
plt.plot(fpr_test, tpr_test, label=f'Test AUC: {roc_auc_test:.2f}')
plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curves')
plt.legend()
plt.grid()
plt.show()
```

```
Training Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00       216
           1       1.00      1.00      1.00       125

    accuracy                           1.00       341
   macro avg       1.00      1.00      1.00       341
weighted avg       1.00      1.00      1.00       341

Validation Classification Report:
              precision    recall  f1-score   support

           0       0.97      0.93      0.95        70
           1       0.89      0.95      0.92        44

    accuracy                           0.94       114
   macro avg       0.93      0.94      0.94       114
weighted avg       0.94      0.94      0.94       114

Test Classification Report:
              precision    recall  f1-score   support

           0       0.96      0.97      0.97        71
           1       0.95      0.93      0.94        43

    accuracy                           0.96       114
   macro avg       0.96      0.95      0.95       114
weighted avg       0.96      0.96      0.96       114
```



Findings:

After retraining the model using the top 10 selected features, I evaluated its performance through classification metrics and ROC curves. The training set achieved perfect precision, recall, and F1-scores, indicating no misclassifications. The validation set showed **a strong overall accuracy of 94%**, with slight decreases in recall and precision for the benign and malignant classes compared to training. The test set maintained **a high overall accuracy of 96%**, demonstrating strong generalization.

The ROC curves further confirmed this performance. The training AUC was **1.00**, while the validation and test AUC scores were **0.97 and 0.99**, respectively, indicating excellent separability between classes even on unseen data. Overall, the model maintained high predictive performance after feature reduction, with only minor trade-offs compared to the original full-featured model in Part 3. Reducing the feature set simplified the model without significantly harming its ability to classify tumors accurately.

Conclusion and Overall Comparisons

In this project, I followed the same overall steps for both Part 3 and Part 4: data preparation, model training, evaluation with confusion matrices, calculation of classification metrics, and plotting of ROC curves. In Part 3, I trained the model using all **30 available features**, achieving **100% training accuracy**, **97.37% validation accuracy**, and **96.49% test accuracy**. The ROC AUC scores were **1.00** across training, validation, and testing. In the confusion matrices for Part 3, the training set had **216 true negatives** and **125 true positives** with no misclassifications, while the validation set had **67 true negatives**, **43 true positives**, **3 false positives**, and **1 false negative**. The test set had **70 true negatives**, **41 true positives**, **1 false positive**, and **4 false negatives**.

In Part 4, after applying feature selection algorithms (Fisher Score and Chi-Square) and reducing the dataset to the **top 10 features**, I repeated the same modeling process. The model achieved **100% training accuracy**, **94% validation accuracy**, and **96% test accuracy**. The AUC scores were **1.00** for training, **0.97** for validation, and **0.99** for testing. In the confusion matrices for Part 4, the training set again had **216 true negatives** and **125 true positives** with no misclassifications. The validation set showed **65 true negatives**, **42 true positives**, **5 false positives**, and **2 false negatives**, while the test set showed **69 true negatives**, **40 true positives**, **2 false positives**, and **3 false negatives**.

As expected, validation performance dropped slightly, by about **3.37%** (from **97.37%** to **94%**), and validation AUC decreased from **1.00** to **0.97**, while test AUC dropped from **1.00** to **0.99**. Despite these minor trade-offs, the overall test accuracy remained nearly identical at **96%**, and the model retained strong generalization ability. Although the number of false positives and false negatives slightly increased in the validation and test sets after feature reduction, the overall impact on predictive performance was minimal.

Feature selection successfully reduced the number of features by two-thirds (from **30** to **10**) without significantly harming classification ability. This project demonstrated that thoughtful feature selection can simplify a model and improve efficiency while maintaining high predictive power, which is critical in applications like medical diagnosis.